

Thesis Project

Building an Analytical Data Warehouse

Project Goal

The main goals of the project are:

- to create a data warehouse using a multi-layer architecture (NDS and DDS);
- develop ETL processes for loading, processing, and analyzing data;
- build data marts for visualizing key metrics of store performance.

The objective of this work is to demonstrate a grasp of key concepts in the subject area and skills in working with essential tools that are in demand today in the field of data engineering.

Project Tasks

1. **Data Processing:**
 - Obtaining raw data from CSV files
 - Data cleaning and normalization
2. **Development of the Data Warehouse structure:**
 - NDS: normalized model for reliable data storage
 - DDS: creation of fact and dimension tables for analytics (star schema)
 - Data Quality: data quality checks
 - Metadata: monitoring the execution of ETL processes
3. **Creating ETL Processes:**
 - Developing processes for loading data from “stage” layer to NDS and DDS layers
 - Orchestrating ETL processes
4. **Data Visualization:**
 - Building dashboards to display key metrics
 - Implementing dashboards for monitoring data quality (metadata and load statuses)

Technologies Stack

- **PostgreSQL 12.9** – DBMS for implementing the data warehouse
- **Python (pandas, faker) 3.11.9** – for processing and generating daily store reports
- **Apache Airflow 2.10.2** – for automating and orchestrating ETL processes
- **Docker 27.2.0** – for containerizing services
- **PowerBI Desktop 2.137.952.0 64-bit** – for creating interactive dashboards
- **dbt Core 1.8.8 + Postgres plugin 1.8.2** – utilized as additional ETL tool

Table of Contents

Project Goal	1
Project Tasks.....	1
Technologies Stack.....	1
Project Phases.....	4
Obtaining Raw Data	5
Daily Reports Generation based on raw data	6
1. Tasks of the Section.....	6
2. Technology Stack and Process Description.....	6
3. New data generation.....	6
Development of Database Schemas as Part of Data Warehouse Design	8
Formation of the Staging Layer of the Data Warehouse.....	9
stage.kaggle_sales	9
stage.kaggle_sales_ready.....	9
stage.days_off.....	10
Formation of the NDS Layer of the Data Warehouse.....	11
nds.orders.....	11
nds.payments	11
nds.payment_types	12
nds.cities	12
nds.branches	12
nds.branches_cities	12
nds.product_lines.....	12
nds.products.....	13
nds.customer_types	13
nds.customers	13
Formation of the DDS Layer of the Data Warehouse.....	14
dim.dates.....	14
fact.sales_items.....	14
dim.branches.....	15
dim.cities	15
dim.branches_cities	15
dim.functional_structure.....	15
dim.products	16
dim.payment_types.....	16
dim.customers.....	16
Formation of the Data Quality Layer	17
dq.stage_kaggle_sales	17
dq.nds_orders.....	17
dq.nds_invoice_customers.....	18
dq.dds_trends.....	18
Formation of the Metadata Layer	19

metadata.etl_status.....	19
Building Dashboards	20
1. Tasks of the Section.....	20
2. BI Tool Used.....	20
3. Data Source	20
4. Model.....	20
5. Visualizations	22
Building ETL Processes and Orchestration	25
1. Tasks of the Section.....	25
2. Process description	25
2.1. Data quality check (DQ) in the stage layer:.....	25
2.2. Loading Data into the NDS Layer:.....	26
2.3. Logging the Load to the NDS Layer	26
2.4. Loading Data into the DDS Layer:.....	26
2.6. Updating Statuses in the Metadata Table.....	27
2.7. Parallel update of Power BI dashboards.....	27
3. ETL process Topology	27
4. Possible enhancements and improvements.....	28
5. Data that did not pass quality checks.....	28
Data Transformation with dbt.....	29
1. Tasks of the Section.....	29
2. Connection Setup.....	29
3. DAG Setup / Managing Model Dependencies	30
4. Model Descriptions	30
5. Snapshot Descriptions	31
6. Creating Documentation.....	31
Results and Conclusions.....	33

Project Phases

1. Obtaining raw data
2. Generation daily reports based on the raw data
3. Schemas development as part of data warehouse design
4. Creation the 'stage' layer of the data warehouse
5. Creation the NDS layer of the data warehouse
6. Creation the DDS layer of the data warehouse
7. Creation the data quality layer
8. Creation the metadata layer
9. Building dashboards
10. Building ETL processes and orchestration

Obtaining Raw Data

The raw dataset is available online at:

<https://www.kaggle.com/datasets/aungpyaeap/supermarket-sales>

This dataset, in CSV format, contains information about sales from three stores of a unified retail chain located in three cities.

The dataset consists of 1000 rows and 17 columns:

- Invoice id: Computer generated sales slip invoice identification number
- Branch: Branch of supercenter (3 branches are available identified by A, B and C).
- City: Location of supercenters
- Customer type: Type of customers, recorded by Members for customers using member card and Normal for without member card.
- Gender: Gender type of customer
- Product line: General item categorization groups - Electronic accessories, Fashion accessories, Food and beverages, Health and beauty, Home and lifestyle, Sports and travel
- Unit price: Price of each product in \$
- Quantity: Number of products purchased by customer
- Tax: 5% tax fee for customer buying
- Total: Total price including tax
- Date: Date of purchase (Record available from January 2019 to March 2019)
- Time: Purchase time (10am to 9pm)
- Payment: Payment used by customer for purchase (3 methods are available – Cash, Credit card and Ewallet)
- COGS: Cost of goods sold
- Gross margin percentage: Gross margin percentage
- Gross income: Gross income
- Rating: Customer stratification rating on their overall shopping experience (On a scale of 1 to 10)

Daily Reports Generation based on raw data

1. Tasks of the Section

Since the data from the raw dataset is insufficient to simulate the admission of new data, a data generator was written. This generator creates data identical in form and content to the original dataset, adding approximately ~ 1200 new rows daily to the staging layer.

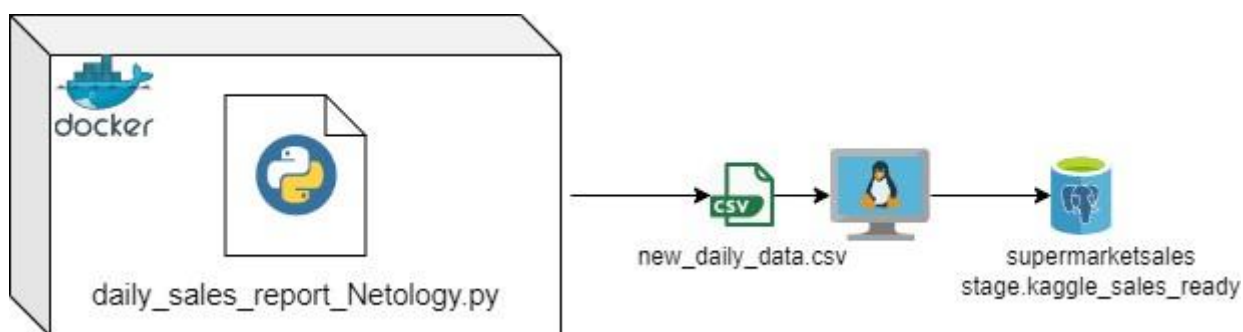
To test the data quality mechanism, a certain percentage of corrupted data is intentionally included, which does not meet the data quality requirements.

2. Technology Stack and Process Description

Data generation is implemented through a **Python script**. For educational purposes and in order to create conditions similar to real-life scenarios, the script is placed in a **Docker container**; the data is exported **via a volume** accessible on the external host.

The artifact is a **CSV file** named 'new_daily_data.csv', which is then processed by a **stored procedure in PostgreSQL**, after which the data is moved to the "stage" layer in the table `stage.kaggle_sales_ready`. The execution of the stored procedure is **triggered through a .bat script** named `call_procedure_sql.bat`.

For educational purposes, the **Ubuntu crontab** is used as the scheduler. It is worth noting that the other orchestration tasks in this project are handled using **Apache Airflow**.



3. New data generation

Data is generated in the Docker container via the script `daily_sales_report_Netology.py`. In the crontab, the script is scheduled to run daily at 7:00 AM. Using a volume mounted through the `HOST_PATH` variable, the output artifact of the script is transferred from the container to the host, specifically to the directory `D:\`.

Further, a .bat script named `call_procedure_sql.bat` copies the artifact to the PostgreSQL directory on the `C:\` drive since the `COPY FROM` command for loading data into the database could only be executed from this directory. It then initiates the execution of the stored procedure `stage.load_raw_data()` in PostgreSQL, which directly loads the data into the database.

The second command in the crontab serves as the scheduler for executing `call_procedure_sql.bat`.

Thus, according to the schedule:

1. New data is generated for our data warehouse.
2. The data is loaded into the stage layer using the 'call_procedure_sql.bat' script, which in turn calls the stored procedure 'load_raw_data()'.

It is important to note that within the aforementioned `call_procedure_sql.bat`, a separate SQL script `call_procedure_script.sql` is invoked, which simply executes the command `CALL stage.load_raw_data()`;

Adjusting the crontab:

```
GNU nano 6.2 /tmp/crontab.IJABnx/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
00 07 * * * /usr/bin/docker start daily_report
03 07 * * * /mnt/c/windows/System32/cmd.exe /c "D:\Курсы и тренинги\Data Engineer Нетология\Дипломная работа\dataset\call_procedure_sql.bat" >> /mnt/d/cron.log 2>&1
```

Help Write Out Where Is Cut Execute Location Read 25 lines Set Mark To Bracket Previous Back Prev Word Home
Exit Read File Replace Paste Justify Go To Line Undo Redo Copy Where Was Next Forward Next Word End

Development of Database Schemas as Part of Data Warehouse Design

The database (hereinafter referred to as the DB) **supermarketsales** simulates a data warehouse (hereinafter referred to as the DW) according to the Inmon model, featuring a three-layer architecture.

List of Database Schemas:

No.	Schema Name	Purpose of the Schema
1	stage	Simulates the raw layer of the DW
2	nds	Simulates the normalized layer of the DW
3	dim	Simulates the data mart layer
4	fact	Simulates the data mart layer
5	dq	Data quality control framework
6	metadata	Metadata framework

Formation of the Staging Layer of the Data Warehouse

In this layer, the initial placement of data occurs during loading from external sources. To speed up INSERT operations, there are no indexes on the tables.

stage.kaggle_sales

Data from the original dataset and subsequent daily reports initially enters this table in AS IS format, without any transformations.

Attribute	Type	Description
invoice_id	varchar(16)	Invoice identification number for payment
branch	varchar(1)	Branch of the supermarket
city	varchar(16)	City of the supermarket location
customer_type	varchar(8)	Customer type based on loyalty program affiliation
gender	varchar(8)	Gender of the customer
product_line	varchar(32)	Product category
unit_price	float4	Price of the product per unit
quantity	int4	Number of products purchased by the customer
tax_5	float4	5% tax fee for customers
total	float4	Total purchase amount including tax
date_	varchar(16)	Purchase date
time_	varchar(8)	Purchase time
payment	varchar(16)	Payment method used by the customer for purchase
cogs	float4	Revenue excluding tax
gross_margin_percentage	float4	Gross margin percentage
gross_income	float4	Gross income
rating	float4	Customer satisfaction rating

stage.kaggle_sales_ready

Data type conversions occur here, and a technical column **check_status** and a synthetic column **product_id** are added.

Attribute	Type	Description
invoice_id	varchar(16)	Invoice identification number for payment
branch	varchar(1)	Branch of the supermarket
city	varchar(16)	City of the supermarket location
customer_type	varchar(8)	Customer type based on loyalty program affiliation
gender	varchar(8)	Gender of the customer
product_line	varchar(32)	Product category
unit_price	numeric(10, 2)	Price of the product per unit
quantity	int4	Number of products purchased by the customer
tax_5	numeric(10, 2)	5% tax fee for customers
total	numeric(18, 2)	Total purchase amount including tax
date_	date	Purchase date
time_	varchar(20) *	Purchase time
payment	varchar(16)	Payment method used by the customer for purchase
cogs	numeric(18, 2)	Revenue excluding tax
gross_margin_percentage	float4	Gross margin percentage
gross_income	numeric(10, 2)	Gross income

rating	numeric(4, 1)	Customer satisfaction rating
check_status	int4	Data quality check flag based on data quality algorithms
product_id	int4	Product identifier; artificially introduced into the dataset for integrity and consistency in subsequent calculations and transformations.

* The data type for time_ is intentionally left as **varchar(20)** to demonstrate the functioning of data quality mechanisms; while using the **time** data type is acceptable, in that case, low-quality data would simply not be processed by the detection algorithms. Moreover, the import process for new report data would be interrupted, requiring an error handler regardless.

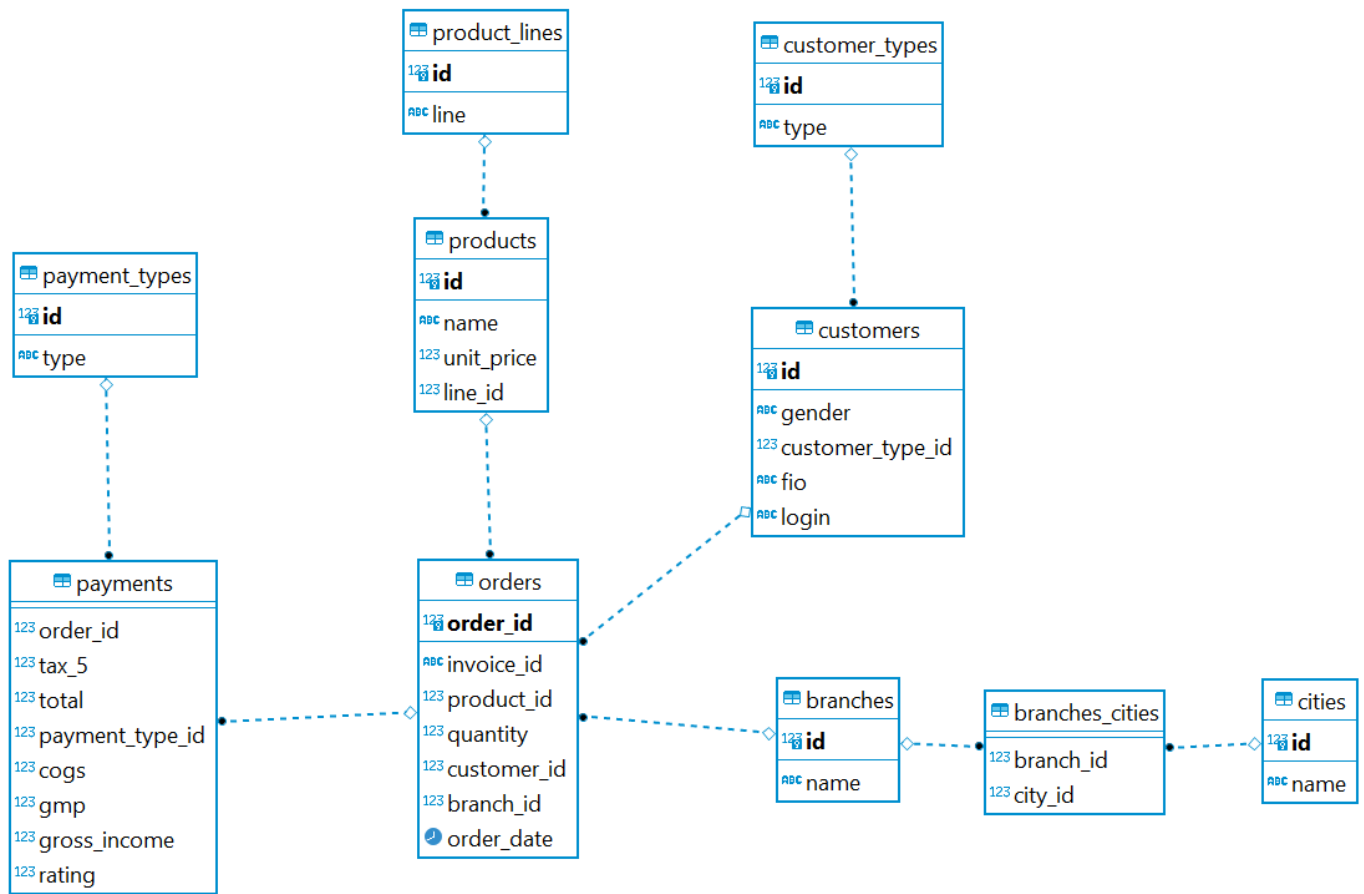
stage.days_off

This dataset imports information about weekends, holidays, pre-holidays, and non-working days.

Attribute	Type	Description
date	varchar(10)	Date in text format
type	varchar(20)	Type of date: weekend/holiday, pre-holiday/non-working day (during COVID-19)

Formation of the NDS Layer of the Data Warehouse

In this layer, the data is normalized and brought in line with the requirements of the Third Normal Form (3NF).



nds.orders

Orders table.

Attribute	Type	Description
order_id	serial4	Order identification number
invoice_id	varchar(16)	Invoice identification number for payment
product_id	int4	Product identification number
quantity	int4	Quantity of the product in the order
customer_id	int4	Customer identification number
branch_id	int4	Branch identification number
order_date	timestamp	Order creation date

The nds.orders table has a PRIMARY KEY on the **order_id** attribute and three FOREIGN KEYS: **product_id**, **customer_id**, and **branch_id**.

To enhance the performance of queries on the **branch_id**, **customer_id**, and **invoice_id** attributes, indexes have been created.

nds.payments

Payments table.

Attribute	Type	Description
order_id	int4	Order identification number
tax_5	decimal(10, 2)	5% tax

total	decimal(18, 2)	Total amount
payment_type_id	int4	Payment type identification number
cogs	decimal(18, 2)	Cost of goods sold
gmp	float4	Gross margin percentage
gross_income	decimal(10, 2)	Gross profit
rating	decimal(4, 1)	Rating

The `nds.payments` table has two FOREIGN KEYs: `order_id` and `payment_type_id`. A covering index `payments_idx_branches` is created for `order_id`, including fields `tax_5`, `total`, `cogs`, and `gross_income`, while for `payment_type_id`, the index `payments_idx_payment_types` is established.

nds.payment_types

Payment types table.

Attribute	Type	Description
id	serial	Payment type identification number
type	varchar(30)	Name of the payment type

The `nds.payment_types` table has a PRIMARY KEY on the `id` attribute.

nds.cities

Cities table.

Attribute	Type	Description
id	serial	City identification number
name	varchar(100)	Name of the city

The `nds.cities` table has a PRIMARY KEY on the `id` attribute.

nds.branches

Branches Table.

Атрибут	Тип	Описание
id	serial	Идентификатор филиала
name	varchar(2)	Название филиала

The `nds.branches` table has a PRIMARY KEY on the `id` attribute.

nds.branches_cities

Branches-Cities relationship table.

Attribute	Type	Description
branch_id	int	Branch identification number
city_id	int	City identification number

The `nds.branches_cities` table has two FOREIGN KEYs: `branch_id` and `city_id`.

nds.product_lines

Product categories table

Attribute	Type	Description
id	serial	Category identification number
line	varchar(100)	Name of the category

The `nds.product_lines` table has a PRIMARY KEY on the `id` attribute.

nds.products

Products table.

This is a synthetic reference table created for data consistency and integrity in the project..

Attribute	Type	Description
id	serial	Product identification number
name	varchar(100)	Name of the product
unit_price	decimal(10, 2)	Price per unit of the product
line_id	int	Product category identification number

The nds.products table has a PRIMARY KEY on the **id** attribute and a FOREIGN KEY on **line_id**. An index **n_idx** is also created for **line_id**.

nds.customer_types

Customer types table.

Attribute	Type	Description
id	serial	Customer type identification number
type	varchar(100)	Name of the customer type

The nds.customer_types table has a PRIMARY KEY on the **id** attribute.

nds.customers

Customers table.

Another synthetic reference table. The data was generated using the Faker library in Python.

Attribute	Type	Description
id	serial	Customer identification number
gender	varchar(6)	Gender of the customer
customer_type_id	int	Customer type identification number
fio	varchar(200)	Full name of the customer
login	varchar(50)	Customer login

The nds.customers table has a PRIMARY KEY on the **id** attribute and a FOREIGN KEY on **customer_type_id**. An index **customers_idx** is created for **customer_type_id**.

Formation of the DDS Layer of the Data Warehouse

The dim and fact schemas together represent the DDS layer. In this layer, data is stored in a format convenient for building data marts. The schema topology is a star schema.

dim.dates

Dates table.

Attribute	Type	Description
id	serial	Date identification number in the format YYYYMMDD
date	date	Date
ansi_date	varchar(10)	Date in the format YYYY-MM-DD
day	int4	Day of the week (1-7)
week_number	int4	Week number in the year
month	int4	Month number (1-12)
year	int4	Year
week_day	int4	Workday flag
holiday	boolean	Holiday flag
preholiday	boolean	Pre-holiday flag
nowork	boolean	Non-working day flag (during COVID-19)

The dim.dates table has a PRIMARY KEY on the id attribute and is clustered on this index.

To improve query performance, the following indexes are also created:

- **dates_dt_idx** on the **date** column
- **month_dt_idx** on the **month** column
- **year_dt_idx** on the **year** column

fact.sales_items

Sales items table.

Attribute	Type	Description
date_key	int	Date identification number
transaction_date	timestamp	Date and time of the order
order_id	int	Order identification number
customer_key	int	Customer identification number
payment_type_id	int	Payment type identification number
branch_id	int	Branch identification number
city_id	int	City identification number
product_key	int	Product identification number
quantity	int	Quantity of products sold
cogs	decimal(18, 2)	Revenue
tax_5	decimal(10, 2)	5% tax
total	decimal(18, 2)	Total sales amount
gmp	real	Gross margin percentage
gross_income	decimal(10, 2)	Gross profit
rating	decimal(4, 1)	Customer satisfaction rating

The fact.sales_items table has several FOREIGN KEYS:

- **date_key** references **dim.dates(id)**;
- **order_id** references **nds.orders(order_id)**;

- **customer_key** references **dim.customers(id)**;
- **payment_type_id** references **dim.payment_types(id)**;
- **product_key** references **dim.products(id)**

To enhance query performance, the following indexes are created:

- Covering index (**cogs, tax_5, total, gross_income**) on the columns (**date_key, order_id**) as **fact_idx_date_**
- **fact_idx_trans_date** on the **transaction_date** column
- **fact_idx_date_order_id** on the **order_id** column
- **fact_idx_date_cust_key** on the **customer_key** column
- **fact_idx_date_ptype_id** on the **payment_type_id** column
- **fact_idx_date_prod_key** on the **product_key** column
- **fact_idx_date_rating** on the **rating** column
- **year_dt_idx** on the **year** column

dim.branches

Branches reference table.

Attribute	Type	Description
id	int	Branch identification number
name	varchar(100)	Name of the branch

The dim.branches table has a PRIMARY KEY on the **id** attribute.

An index **dim_idx_branches** is created on the **id** column.

dim.cities

Cities reference table

Attribute	Type	Description
id	int	City identification number
name	varchar(100)	Name of the city

The dim.cities table has a PRIMARY KEY on the **id** attribute.

An index **dim_idx_cities** is created on the **id** column..

dim.branches_cities

Branches-Cities relationship table

Attribute	Type	Description
branch_id	int	Branch identification number
city_id	int	City identification number

The dim.branches_cities table has two FOREIGN KEYs:

- **branch_id** references **dim.branches(id)**
- **city_id** references **dim.cities(id)**

dim.functional_structure

Functional structure reference table

Attribute	Type	Description
branch_id	int	Branch identification number
branch	varchar(2)	Name of the branch
city_id	int	City identification number
city	varchar(100)	Name of the city

The dim.functional_structure table has a PRIMARY KEY on the **branch_id** and **city_id** attributes.
An index **dim_idx_fs** is created on the **city_id** column.

dim.products

Products reference table

Attribute	Type	Description
id	int	Product identification number
product_name	varchar(100)	Name of the product
unit_price	decimal(10, 2)	Price per unit of the product
product_line	varchar(100)	Product category

The dim.products table has a PRIMARY KEY on the id attribute.
An index **dim_idx_products** is created on the **id** column.

dim.payment_types

Payment types reference table

Attribute	Type	Description
id	int	Payment type identification number
payment_type	varchar(30)	Name of the payment type

The dim.payment_types table has a PRIMARY KEY on the id attribute.
An index **dim_idx_payment_types** is created on the **id** column.

dim.customers

Customers reference table.

Attribute	Type	Description
id	serial	Customer identification number in the DDS layer
customer_id	int	Customer identification number in the NDS layer
customer_type	varchar(100)	Customer type based on the loyalty program
gender	varchar(6)	Gender of the customer
fio	varchar(200)	Full name of the customer
login	varchar(50)	Customer login
active_from	int	Date of customer registration in the system
active_to	int	Date of customer removal from the system / data modification

The dim.customers table has a PRIMARY KEY on the **id** attribute.
This relationship is the Slowly Changing Dimension (SCD) Type 2.

The following indexes have been created:

- Covering index (**gender, fio, login**) **dim_idx_customers_main** on the **id** column
- **dim_idx_customers_id** on the **customer_id**
- **dim_idx_active** on the columns (**active_from, active_to**)

Formation of the Data Quality Layer

The dq schema contains data that did not pass the quality checks.

dq.issues

Issues Reference Table

Attribute	Type	Description
id	serial	Issue identification number
desc	varchar(200)	Description of the issue

The dq.issues table has a PRIMARY KEY on the **id** attribute.

dq.stage_kaggle_sales

Table of poor quality data identified in the stage layer

Attribute	Type	Description
invoice_id	varchar(16)	Invoice identification number for payment
branch	varchar(1)	Branch of the supermarket
city	varchar(16)	City of the supermarket location
customer_type	varchar(8)	Customer type based on loyalty program affiliation
gender	varchar(8)	Gender of the customer
product_line	varchar(32)	Product category
unit_price	numeric(10, 2)	Price of the product per unit
quantity	int	Number of products purchased by the customer
tax_5	numeric(10, 2)	5% tax fee for customers
total	numeric(18, 2)	Total purchase amount including tax
date	date	Purchase date
time	varchar(20)	Purchase time
payment_type	varchar(16)	Payment method used by the customer for purchase
cogs	numeric(18, 2)	Revenue excluding tax
gross_margin_percentage	float4	Gross margin percentage
gross_income	numeric(10, 2)	Gross income
rating	numeric(4, 1)	Customer satisfaction rating
issue_id	int	Issue identification number

The dq.stage_kaggle_sales table has a FOREIGN KEY on the **issue_id**.

dq.nds_orders

Table of poor quality data identified in the nds layer.

This table contains anomalous data for days where the number of orders significantly exceeds the daily norm, as well as identified problematic cases where the revenue reported in the original dataset in the 'cogs' column differs from the revenue calculated in the nds.orders table by multiplying the 'quantity' and 'unit_price' attributes.

Attribute	Type	Description
order_date	date	Order date
num_orders	int	Number of orders
cogs_from_nds	decimal(18, 2)	Revenue from NDS
cogs_check_sum	decimal(18, 2)	Revenue from staging
issue_id	int	Issue identification number

he dq.nds_orders table has a FOREIGN KEY on the **issue_id**.

dq.nds_invoice_customers

Table of poor quality data identified in the nds layer.

This table contains anomalous cases where there is more than one customer per order.

Attribute	Type	Description
invoice_id	varchar(16)	Invoice identification number for payment
order_date	timestampz	Order date
num_customers	int8	Number of customers attached to the order
issue_id	int	Issue identification number

The dq.nds_invoice_customers table has a FOREIGN KEY on the **issue_id**.

dq.dds_trends

Trends table.

This table contains aggregated data on the average number of orders per day and the daily total of orders.

Attribute	Type	Description
check_date	date	Date of the check
avg_num_orders	int	Average number of orders
delta_avg_num_orders	int	Change in the average number of orders
avg_total	int	Average total of orders
delta_avg_total	int	Change in the average total of orders

The dq.dds_trends table has no constraints or indexes.

Formation of the Metadata Layer

The metadata schema contains metadata about the status of ETL processes and the number of affected rows.

metadata.etl_status

ETL status table.

Attribute	Type	Description
id	serial	Record identification number in the table
dag_id	text	DAG identification number
task_id	text	Task identification number
execution_date	timestamp	Date and time of task_id execution
status	text	Status of task_id execution
rows_processed	int	Number of rows processed in task_id
target_table	text	Target table where data was loaded
last_row_count	int	Total number of rows in the target table at the time of the record creation in metadata.etl_status
created_at	timestamp	Date and time of record creation

The metadata.etl_status table has a PRIMARY KEY on the **id** attribute.

The following indexes have been created:

- **md_idx_date_load** on the **execution_date** column
- **md_idx_date_target** on the **target_table** column

Building Dashboards

1. Tasks of the Section

To demonstrate successful integration of BI tools with the data mart layer (DDS) in the data warehouse.

2. BI Tool Used

Microsoft PowerBI Desktop 2.137.952.0 64-bit

3. Data Source

The data source is the **supermarketsales** DB, specifically the fact, dim, and metadata schemas. The connection is established through the native connector for PostgreSQL.

Настройки источника данных

Управление параметрами для источников данных, подключенных для использования Power BI Desktop.

☒ Источники данных в текущем файле ☐ Глобальные разрешения

Поиск по параметрам источников данных

localhost:5432;supermarketsales

Изменить источник...

Экспорт: PBIDS

Править разрешения...

Очистить разрешения ▾

Закреть

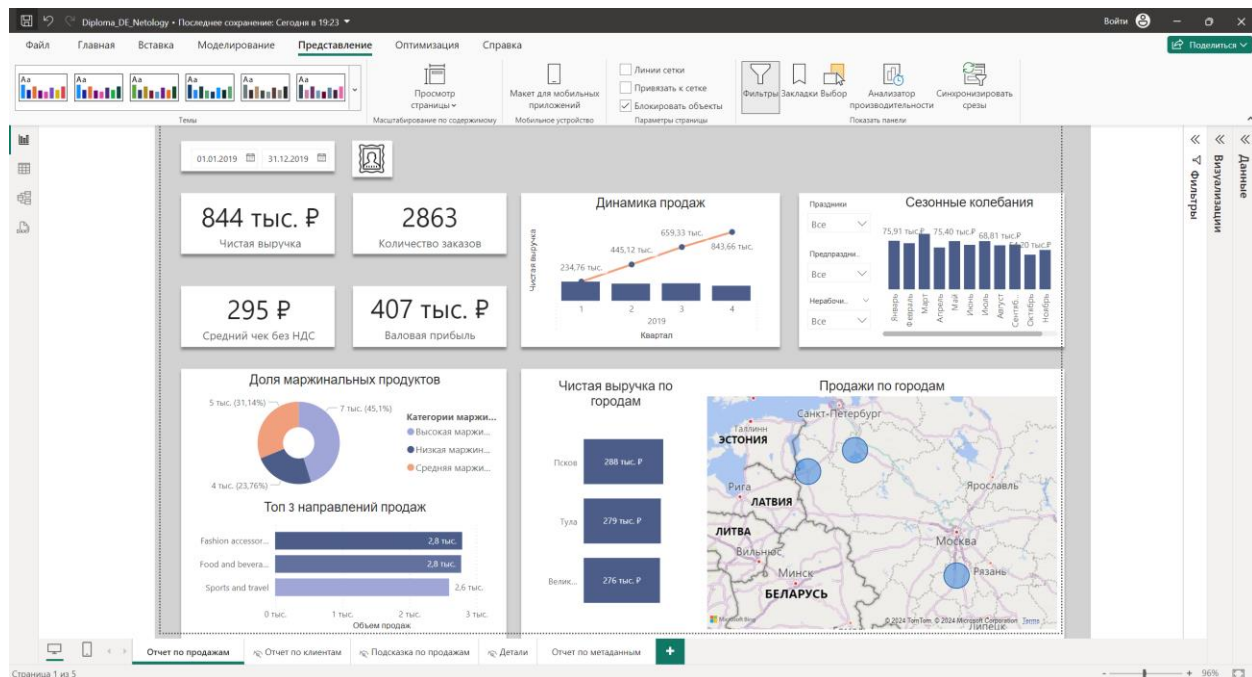
4. Model

- One-to-many (1:*) relationships are established between the dimension tables and the fact table. There are also one-to-one (1:1) relationships between the dates table and the tables for holidays, pre-holidays, and non-working days;
- the dates table is defined;
- data types for the columns are configured, and display settings for columns and measures in the visualizations are adjusted;
- the necessary measures are prepared.



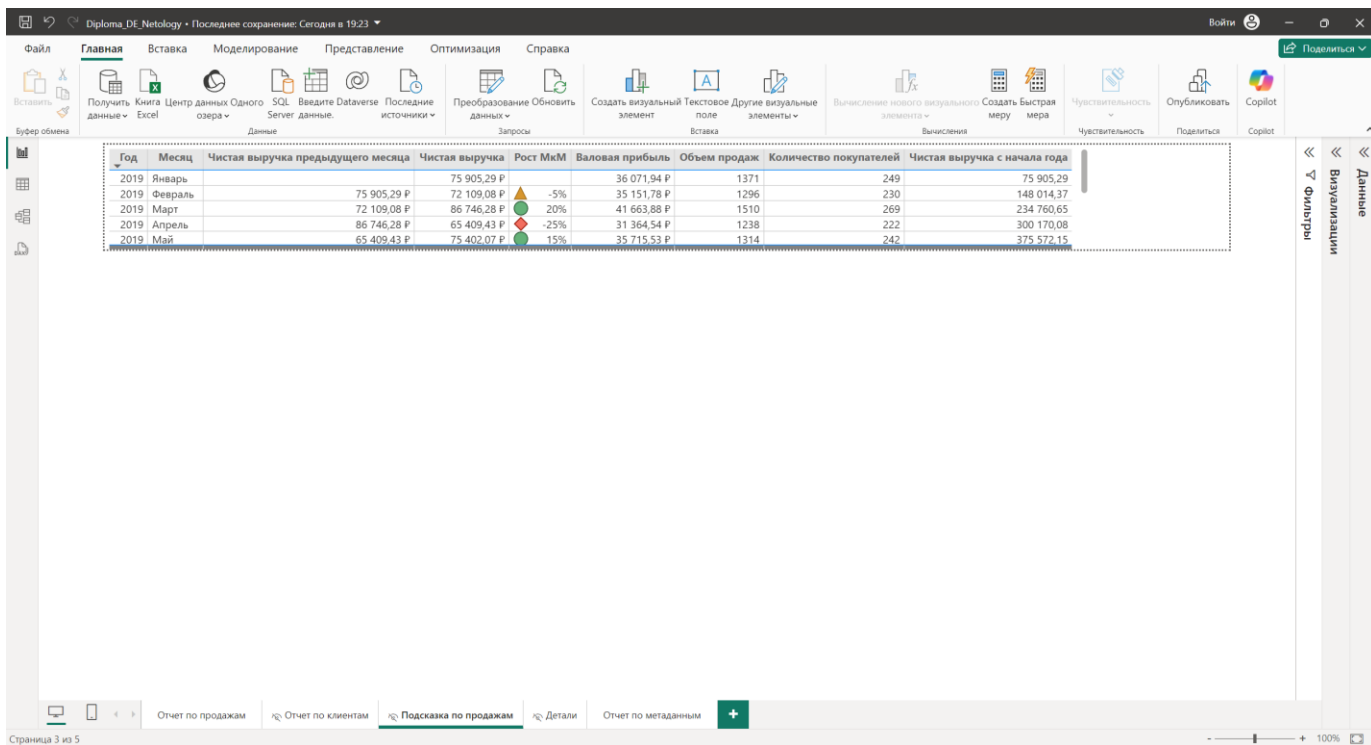
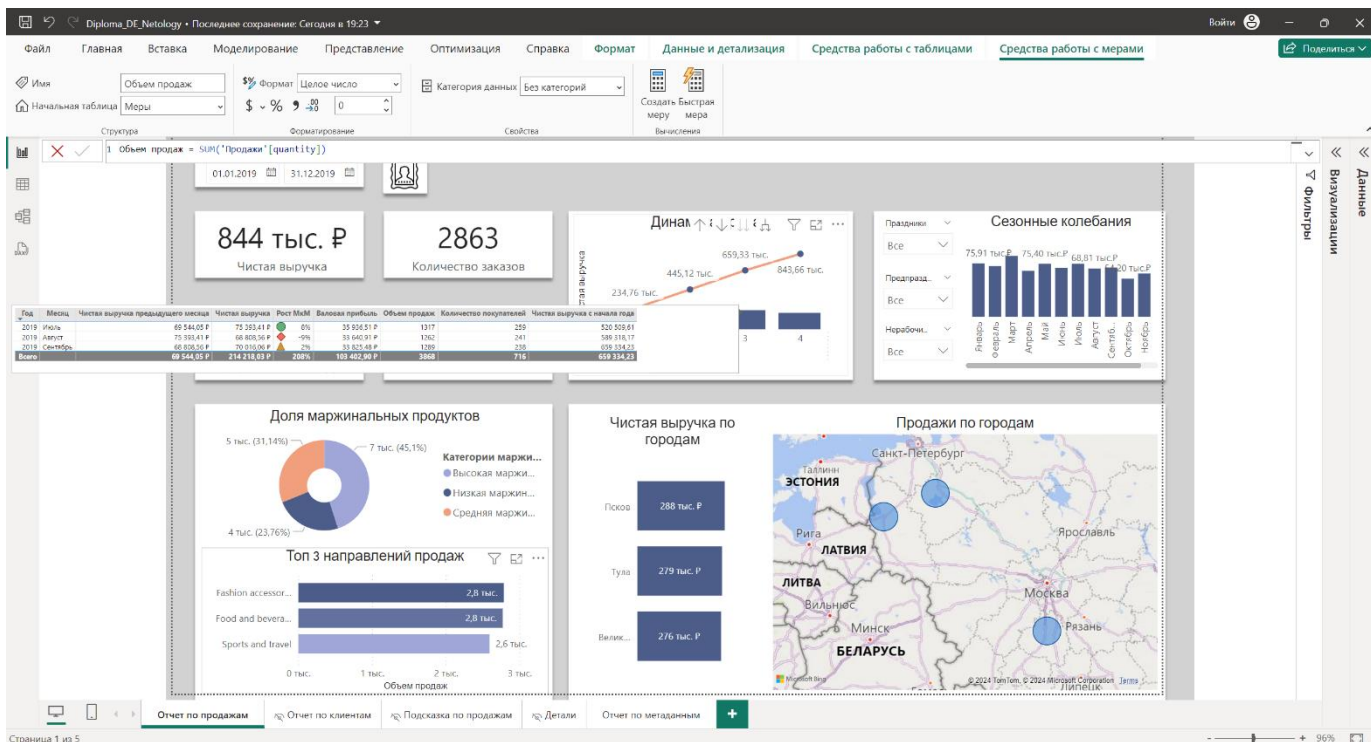
5. Visualizations

The main dashboard – «Sales Report»

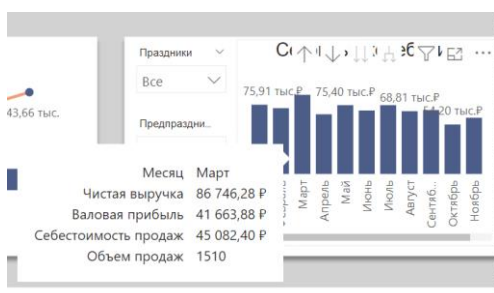


Key Components:

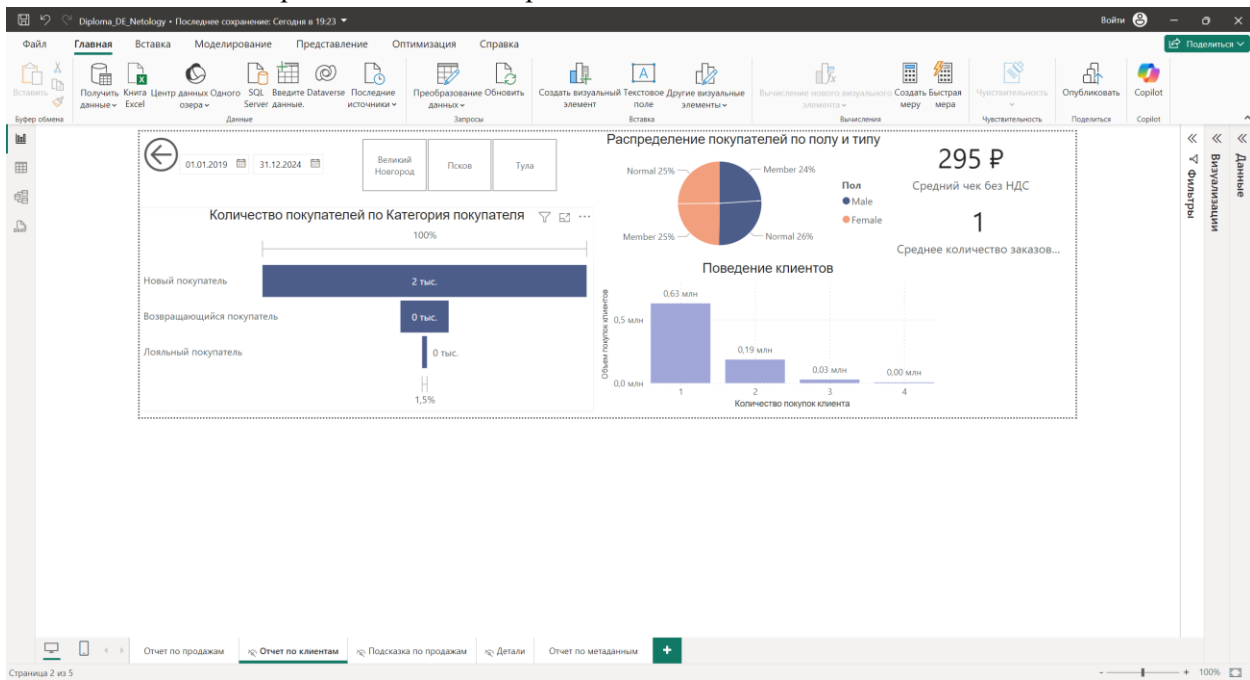
- Overall Metrics:
 - Total Sales:** Quantity of products sold
 - Number of Orders:** Total number of orders
 - Average Check:** Average amount of a single order
 - Net Revenue:** Total value of sold products
 - Cumulative Net Revenue since the beginning of the year**
 - Gross Profit:** Difference between total sales and cost of goods sold
- Trends Over Time:
 - Sales dynamics by days/months/years.
 - Seasonal fluctuations: Analysis of peak sales periods related to holidays or promotions..
- Regional Analysis:
 - Sales by cities and branches:
- Product Category Analysis:
 - Top 3 sales directions
 - Product segmentation by margin level
- Customer Profile
- Drill-down and Tooltips:
 - A page is prepared for tooltips, providing extended information in the form of a tabular reference on sales in the given context.



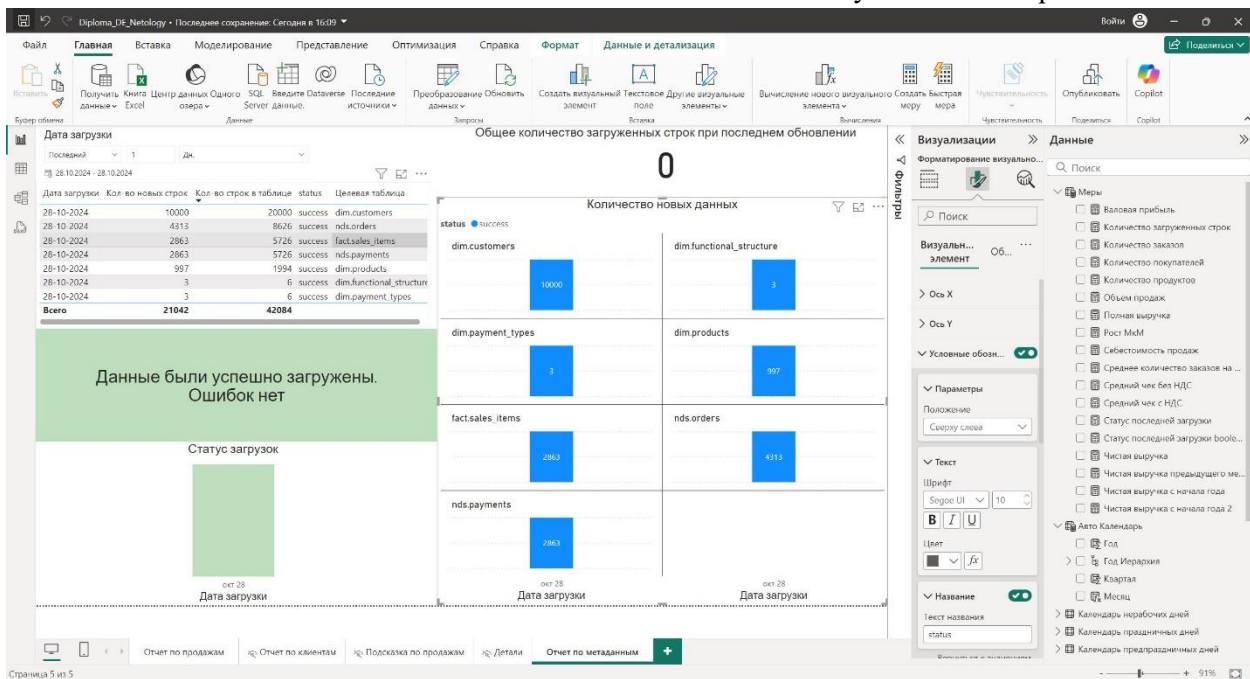
- Additional information is displayed as a tooltip when hovering the cursor over elements:



- A customer profile has been compiled:



- A dashboard with technical information from the metadata layer has been implemented:



Building ETL Processes and Orchestration

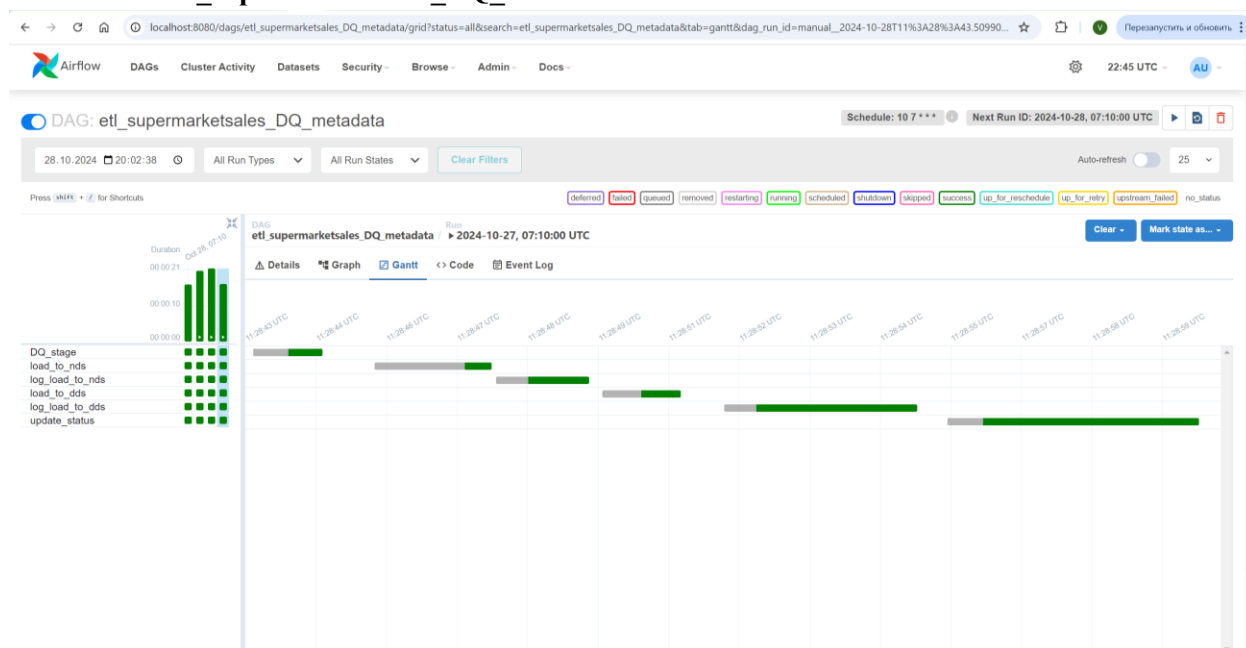
1. Tasks of the Section

To develop and configure ETL processes to ensure reliable and timely delivery of data to the data warehouse. Apache Airflow version 2.10.2, based on Python 3.10.12, has been chosen as the orchestrator and ETL loader, working in conjunction with PostgreSQL 12.9.

To accomplish this task, a single Directed Acyclic Graph (DAG) was developed, providing:

- Quality control, data integrity maintenance, and loading of data from the raw staging layer (stage) to the normalized layer (nds);
- Quality control, data integrity maintenance, and transfer of data from the nds layer to the data mart layer (dds);
- Logging of metadata, such as task execution status and amount of processed information.

DAG Name: **etl_supermarketsales_DQ_metadata**



2. Process description

The ETL process is divided into the following stages::

2.1.Data quality check (DQ) in the stage layer:

- Task name: **DQ_stage**
- Operator used: **PostgresOperator**; executes the SQL script **DQ_stage.sql**
- The staging layer data is enriched with the **product_id** attribute (synthetic data is generated), and the attributes **unit_price**, **tax_5**, **cogs**, **total**, and **gross_income** are updated and recalculated according to the values from the **nds.products** reference table. This is a necessary approach to simulate the normal operation of the data warehouse.
- Further the initial data quality and consistency check is performed:
 - **invoice_id** matches the pattern **###-##-####**
 - **unit_price** > 0
 - **quantity** > 0
 - **tax_5** > 0
 - **time** is within the range from 00:00:00 to 23:59:59

- **rating** is in the range from 0 to 10
- Data that fails the checks is sent to the **dq** schema of the supermarketsales database, specifically to the table **dq.stage_kaggle_sales**, for review by the data engineer or the responsible employee for the source data.
- Data that did not pass the data quality check (DQ) is removed from the **stage.kaggle_sales_ready** table.
- Task result:
 - Data in the **stage.kaggle_sales_ready** table is prepared for transfer to the nds layer.
 - Initial quality control of the data has been performed; data that failed the checks has been sent to **dq.stage_kaggle_sales**

2.2.Loading Data into the NDS Layer:

- Task name: **load_to_nds**
- Operator used: **PostgresOperator**; executes the SQL script **ETL_DQ_nds.sql**
- In the first step, the **dq.nds_orders** and **dq.nds_invoice_customers** tables are cleared, simulating the review of poor-quality data identified in this task
- An anomaly check is performed on the data, specifically checking for extreme values in the number of orders placed in a single day. The threshold for triggering this check is set at a 3-fold increase over the average number of orders per day
- Data that fails the DQ checks is sent to the **dq.nds_orders** table with the identifier **issue_id = 7**, after which the data is cleared from the source in the **stage.kaggle_sales_ready** table.
- A logical check is performed to ensure that one order can only have been processed by one customer.
- Data that fails the DQ checks is sent to the **dq.nds_invoice_customers** table with the identifier **issue_id = 8**, after which the data is cleared from the source in the **stage.kaggle_sales_ready** table.
- Remaining data in **stage.kaggle_sales_ready** is loaded into the nds layer, specifically into the **nds.orders** table and **nds.payments**.
- Task result:
 - Data from the **stage.kaggle_sales_ready** table has been loaded into the **nds.orders** and **nds.payments** tables
 - Data quality checks have been performed; data that failed the checks has been segmented and sent to **dq.nds_orders** and **dq.nds_invoice_customers**;

2.3.Logging the Load to the NDS Layer

- Task name: **log_load_to_nds**
- Operator used: **PythonOperator**; executes the function **log_metadata_initial**
- During the logging process, information such as the DAG name, the name of the previous task, the execution time of the previous task, and the number of rows processed is saved.
- Task result: Metadata about the execution of the **load_to_nds** task is stored in the **metadata** schema, specifically in the **etl_status** table.

2.4.Loading Data into the DDS Layer:

- Task name: **load_to_dds**
- Operator used: **PostgresOperator**; executes the SQL script **ETL_DQ_dds.sql**
- In the first step, the average number of orders at the time of loading data into the DDS layer is recorded, along with the average order amounts for the entire data history.

- **Incremental insertion** of data from the nds layer into the corresponding tables of the dds and fact layers is performed; the **fact.sales_items** table and **dimension tables in the dim schema** are populated with new data.
- The average number of orders is recorded again, but this time considering the newly inserted data.
- Information about the difference between the measured indicators is written to the **dq.dds_trends** table.
It is assumed that the data in the dq.dds_trends table can serve as an indicator in the event of significant deviations of the average values from the trend line.
- Task result:
 - **data** from the nds layer is **loaded into the dds layer**.
The architecture of this data layer is a "star" schema.
 - Aggregate metrics are loaded into the **dq.dds_trends** table for further analysis.

2.5. Logging the Load to the DDS Layer:

- Task name: **log_load_to_dds**
- Operator used: **PythonOperator**; executes the function **log_metadata_initial**
- During the logging process, information such as the DAG name, the name of the previous task, the execution time of the previous task, and the number of rows processed is saved.
- Task result: metadata about the execution of the **load_to_dds** task is stored in the **metadata** schema, specifically in the **etl_status** table.

2.6. Updating Statuses in the Metadata Table

- Task name: **update_status**
- Operator used: **PythonOperator**; executes the function **update_statuses**
- In this step, **the public.task_instance table from the internal Airflow database** is queried to obtain the correct statuses of the previous tasks.
- Task result: metadata about the execution statuses of the **load_to_dds** and **load_to_dds** tasks is stored in the **metadata** schema, specifically in the **etl_status** table.

2.7. Parallel update of Power BI dashboards

This is an optional task that is not implemented in the project.

After the loading into the DDS layer is complete, a task can be added to refresh reports in Power BI.

3. ETL process Topology

- Implemented Topology:

DQ_stage --> load_to_dds --> log_load_to_dds --> load_to_dds --> log_load_to_dds --> update_status

- Possible Improvement:

DQ_stage --> load_to_dds --> log_load_to_dds --> load_to_dds --> log_load_to_dds --> update_status

|
(in parallel)

--> Power BI Data Mart refresh

In this project, it was not possible to implement this, as the use of the Power BI API is only available in the paid version.

4. Possible enhancements and improvements

- Automating the Power BI refresh after the loading process is complete.
- Optimizing SQL scripts and index tuning to enhance performance
- Setting up alerts in case of DAG failures

5. Data that did not pass quality checks

```
110= SELECT issue_id, COUNT(*)
111 FROM dq.stage_kaggle_sales
112 GROUP BY issue_id
113 ORDER BY 1;
```

stage_kaggle_sales 1 ×

SELECT issue_id, COUNT(*) FROM dq.stage_kaggle_sales GROUP BY issue_id

issue_id	count
1	216
2	225
3	118
4	202
5	370
6	204

Словарь (issues):

Значение	Описание
1	incorrect invoice
2	incorrect unit_price
3	incorrect quantity
4	incorrect tax_5
5	incorrect rating
6	incorrect time
7	number of orders is more than 10*daily
8	there are more than 1 customer for 1 invoice_id
9	cogs was not calculated accurately

```
110= SELECT issue_id, COUNT(*)
111 FROM dq.nds_orders no2
112 GROUP BY issue_id
113 ORDER BY 1;
```

nds_orders 1 ×

SELECT issue_id, COUNT(*) FROM dq.nds_orders no2 GROUP BY issue_id

issue_id	count
1	40
2	83

Словарь (issues):

Значение	Описание
1	incorrect invoice
2	incorrect unit_price
3	incorrect quantity
4	incorrect tax_5
5	incorrect rating
6	incorrect time
7	number of orders is more than 10*daily
8	there are more than 1 customer for 1 invoice_id
9	cogs was not calculated accurately

Figures 1-3

```
109= SELECT issue_id, COUNT(*)
110 FROM dq.nds_invoice_customers
111 GROUP BY issue_id
112 ORDER BY 1;
```

nds_invoice_customers 1 ×

SELECT issue_id, COUNT(*) FROM dq.nds_invoice_customers GROUP BY issue_id

issue_id	count
1	141

Словарь (issues):

Значение	Описание
1	incorrect invoice
2	incorrect unit_price
3	incorrect quantity
4	incorrect tax_5
5	incorrect rating
6	incorrect time
7	number of orders is more than 10*daily
8	there are more than 1 customer for 1 invoice_id
9	cogs was not calculated accurately

Data Transformation with dbt

1. Tasks of the Section

dbt was used as an alternative tool for data transformation purposes in an educational context. Its use was not mandatory in this case and led to some duplication of functionality between Airflow and dbt, but the goal was to get hands-on experience with dbt.

Within this project, dbt solves the following tasks:

1. Detects anomalies in the number of orders per day at the model level.
2. Transfers data from the stage layer to the NDS layer (implemented with the example of the dbt.orders table).
3. Creates snapshots of selected relationships, implementing data history (SCD Type 2) (implemented with the example of the dbt.orders table).
4. Generates automatic documentation for the project.

Important and useful features such as automatic data testing and the use of macros were not covered in this project.

2. Connection Setup

Following the official guide's recommendations, a separate virtual environment in Python was created named dbt-env.

Then, dbt-core and the necessary adapter (PostgreSQL) were installed, and the connection to the supermarketsales database was configured in the profiles.yml file:

```
(dbt-env) quantum@DESKTOP-VJG26RT:/mnt/d/Курсы и тренинги/Data Engineer Нетология/Дипломная работа/dbt$ dbt init dbt_netology
07:18:44 Running with dbt=1.8.8
07:18:44 [ConfigFolderDirectory]: Unable to parse dict {'dir': PosixPath('/home/quantum/.dbt')}
07:18:44 Creating dbt configuration folder at
07:18:44
Your new dbt project "dbt_netology" was created!

For more information on how to configure the profiles.yml file,
please consult the dbt documentation here:

  https://docs.getdbt.com/docs/configure-your-profile

One more thing:

Need help? Don't hesitate to reach out to us via GitHub issues or on Slack:

  https://community.getdbt.com/

Happy modeling!

07:18:44 Setting up your profile.
Which database would you like to use?
[1] postgres

(Don't see the one you want? https://docs.getdbt.com/docs/available-adapters)

Enter a number: 1
host (hostname for the instance): localhost
port [5432]: 5432
user (dev username): dbt_user
pass (dev password):
dbname (default database that dbt will build objects in): supermarketsales
schema (default schema that dbt will build objects in): dbt
threads (1 or more) [1]: 1
07:44:30 Profile dbt_netology written to /home/quantum/.dbt/profiles.yml using target's profile_template.yml and your supplied
(dbt-env) quantum@DESKTOP-VJG26RT:/mnt/d/Курсы и тренинги/Data Engineer Нетология/Дипломная работа/dbt$
```

After creating the dbt project (using the dbt init command), it becomes possible to configure the dbt-project.yml:

```
(dbt-env) quantum@DESKTOP-VJG26RT:/mnt/d/Курсы и тренинги/Data Engineer Нетология/Дипломная работа/dbt/dbt_netology$ cat dbt_project.yml
# Name your project! Project names should contain only lowercase characters
# and underscores. A good package name should reflect your organization's
# name or the intended use of these models
name: 'dbt_netology'
version: '1.0.0'

# This setting configures which "profile" dbt uses for this project.
profile: 'dbt_netology'

# These configurations specify where dbt should look for different types of files.
# The 'model-paths' config, for example, states that models in this project can be
# found in the "models/" directory. You probably won't need to change these!
model-paths: ["models"]
analysis-paths: ["analyses"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

clean-targets: # directories to be removed by `dbt clean`
  - "target"
  - "dbt_packages"

# Configuring models
# Full documentation: https://docs.getdbt.com/docs/configuring-models
#
# In this example config, we tell dbt to build all models in the example/
# directory as views. These settings can be overridden in the individual model
# files using the `{{ config(...) }}` macro.
models:
  dbt_netology:
    # Config indicated by + and applies to all files under models/example/
    example:
      +materialized: view
```

The dbt debug command allows to verify the correct connection settings:

```
(dbt-env) quantum@DESKTOP-VJG26RT:/mnt/d/Курсы и тренинги/Data Engineer Нетология/Дипломная работа/dbt/dbt_netology$ dbt debug
14:52:30 Running with dbt=1.8.8
14:52:30 dbt version: 1.8.8
14:52:30 python version: 3.10.12
14:52:30 python path: /home/quantum/dbt-env/bin/python
14:52:30 os info: Linux-5.15.153.1-microsoft-standard-WSL2-x86_64-with-glibc2.35
14:52:30 Using profiles dir at /home/quantum/.dbt
14:52:30 Using profiles.yml file at /home/quantum/.dbt/profiles.yml
14:52:30 Using dbt_project.yml file at /mnt/d/Курсы и тренинги/Data Engineer Нетология/Дипломная работа/dbt/dbt_netology/dbt_project.yml
14:52:30 adapter type: postgres
14:52:30 adapter version: 1.8.2
14:52:30 Configuration:
14:52:30   profiles.yml file [OK found and valid]
14:52:30   dbt_project.yml file [OK found and valid]
14:52:30 Required dependencies:
14:52:30   - git [OK found]

14:52:30 Connection:
14:52:30   host: 10.0.0.5
14:52:30   port: 5432
14:52:30   user: dbt_user
14:52:30   database: supermarketsales
14:52:30   schema: dbt
14:52:30   connect_timeout: 10
14:52:30   role: None
14:52:30   search_path: None
14:52:30   keepalives_idle: 0
14:52:30   sslmode: None
14:52:30   sslcert: None
14:52:30   sslkey: None
14:52:30   sslrootcert: None
14:52:30   application_name: dbt
14:52:30   retries: 1
14:52:30 Registered adapter: postgres=1.8.2
14:52:31 Connection test: [OK connection ok]

14:52:31 All checks passed!
(dbt-env) quantum@DESKTOP-VJG26RT:/mnt/d/Курсы и тренинги/Data Engineer Нетология/Дипломная работа/dbt/dbt_netology$
```

3. DAG Setup / Managing Model Dependencies

Due to the simplicity of the models, a DAG was not created.

However, it is assumed that, if necessary, the **ref** operator will be used to set up the DAG and manage model dependencies.

4. Model Descriptions

The main approach when writing dbt models is to have one DML/DDL operation per model. If necessary, dependencies between models are configured, thereby setting up the DAG.

For each model, a corresponding SQL script is created, which is stored in the **./models** directory.

- **bad_dates**: This model performs the same check as the **load_to_nds** task in Airflow – it checks for anomalous outliers in the number of orders per day. The identified dates are recorded in the

dbt.bad_dates table.

The model uses the materialization parameter `{{ config(materialized='table') }}`

- **orders**. This model performs incremental loading of cleaned data into the dbt.orders table. The model uses the materialization parameter `{{ config(materialized='incremental', unique_key='invoice_id') }}`.

The models are executed with the **dbt run** command.

The screenshot shows a database client interface with a tree view on the left and a query editor on the right. The tree view shows the database structure, including the 'supermarketsales' database and the 'dbt' schema. The 'dbt' schema contains several tables, including 'bad_dates', 'orders', and 'orders_snapshot'. The 'bad_dates' table is highlighted, showing its size as 16K. The query editor on the right shows a SQL query: `SELECT * FROM dbt.bad_dates bd;`. Below the query editor, the results of the query are displayed in a table format. The table has four columns: 'bad_date', 'max_order_limit', and 'count'. The data is as follows:

bad_date	max_order_limit	count
2019-09-11	28	39
2019-06-06	28	28
2019-10-24	28	38
2019-03-29	28	29
2019-05-14	28	29
2019-11-30	28	29
2019-03-08	28	36
2019-08-28	28	34
2019-03-15	28	32
2019-08-14	28	28
2019-08-11	28	37
2019-01-26	28	28
2019-05-16	28	28
2019-09-10	28	31
2019-12-25	28	28
2019-11-07	28	29

5. Snapshot Descriptions

Snapshots are used to create versioning of data, allowing tracking of changes over time while preserving historical records in tables.

In the `./snapshots` folder, one snapshot script (`orders_snapshot.sql`) has been created for the `dbt.orders` table. This snapshot automatically tracks changes in the `product_id` and `customer_id` fields.

The tracking strategy used is **check**.

dbt compares the values of the specified fields in existing snapshot records with new values from the source table. If any field differs, the record will be updated or added.

Another strategy, **timestamp**, could have been used, but in that case, a versioning attribute based on the record's creation time (e.g., the `'updated_at'` field) would need to be added.

6. Creating Documentation

In order to create documentation describing the used models and snapshots, a `models.yml` file was created in the `./models` directory.

Then, the following commands were executed to generate the documentation and start the server to view it:

```
dbt docs generate
```

```
dbt docs serve --port 8081 (port 8080 is occupied by Apache Airflow)
```

← → ↺ 🏠

localhost:8083/#/model/model.dbt_netology.orders

☆ 📄

dbt

Search for models...

Overview

Project

Database

Group

Projects

dbt_netology

models

bad_dates

example

orders

snapshots

orders

incremental

Details

Description

Columns

Referenced By

Depends On

Code

Details

TAGS	OWNER	TYPE	PACKAGE	LANGUAGE	RELATION	ACCESS	VERSION	CONTRACT
untagged	dbt_user	table	dbt_netology	sql	supermarketsales.dbt.orders	protected		Not Enforced

Description

The incremental model for loading orders from table kagg1e_sales_ready

Columns

COLUMN	TYPE	DESCRIPTION	CONSTRAINTS	DATA TESTS	MORE?
invoice_id	character varying(16)	Unique order identifier			>
product_id	double precision	Identifier of the product			>
quantity	integer	Quantity of product in current order			>
customer_id	double precision	Identifier of the client			>
branch_id	integer	Identifier of branch			>
order_date	text	Order date concatenated from attribut...			>

Results and Conclusions

The following tasks were accomplished within the project using various technologies:

No.	Task Completed	Technology Stack
1	A data warehouse was created, and DDL and DML scripts were written for all layers of the data warehouse	PostgreSQL
2	A daily report generator was created, simulating both "correct" data from a DQ perspective and erroneous data	Python Docker
3	Orchestration of daily report loading was set up	Crontab .bat script (SQL script execution) PostgreSQL stored procedure
4	A DAG was created to implement ETL processes for loading data into the data warehouse, validating it, and logging ETL task statuses	Apache Airflow
5	Dashboards were built based on the data mart layer	Power BI
6	dbt models have been configured for an alternative ETL process setup and data quality (DQ) checks	dbt

The construction of the NDS and DDS layers allows for efficient storage and analysis of data, while the creation of dashboards helps visualize key metrics and improve decision-making processes.

The project demonstrates the potential for using ETL processes to automate data loading and provide flexibility in configuration and scaling.

Additionally, data quality control and tracking of uploads are implemented through the metadata layer.